DOCKER

Term	Easy meaning	Technical example
Docker	Platform to package and run applications in containers.	Run Nginx in an isolated environment.
Image	Template used to create containers.	nginx, httpd, ubuntu:latest
Container	An isolated runtime instance of an image.	Container named akash1 running Nginx.
Docker Engine	Software that manages containers, images, networks and volumes.	Docker service running on Ubuntu.
Docker Hub	Online registry to store and share images.	akashdockeraws/akashdocker1008:ubuntuimage
Dockerfile	Text file with instructions for building an image.	FROM ubuntu:latest
Volume	Docker-managed storage that can outlive a container.	akashvolume
Network	Connects containers and controls how they communicate.	akashbridge
Port mapping	Forwards a host port to a container port.	-p 8080:80
Repository / tag	Repository groups image versions; tag identifies a version/variant.	akashdockeraws/akashdocker1008:ubuntuimage
EXPOSE	Documents an intended container port; does not publish it by itself.	EXPOSE 80

2. Basic Linux commands used before Docker

No.	Command	Theory in simple language	Example / expected result	Exam tip
1	sudo su -	Switch to root using a login shell and load root's login environment.	Use on Ubuntu; prompt may change to #.	Root has extensive privileges; use carefully.
2	sudo su	Switch to root without the same full login-environment setup.	Run whoami to verify.	Both commands switch to root; '-' requests a login shell.
3	whoami	Display current	Output may be root.	Checks which

		username.		account is active.
4	pwd	Print current working directory.	May show /root or /home/ubuntu.	pwd = present working directory.
5	cd /home	Move to /home.	Run pwd to verify.	cd = change directory.
6	cd /	Move to the filesystem root directory.	pwd displays /.	/ is the top of the Linux filesystem.
7	ls	List files and directories.	ls /home lists entries in /home.	ls -l gives details.
8	touch akash.txt	Create an empty file if missing; otherwise update timestamps.	touch /home/akash.txt	Does not normally add text.
9	touch dockerfile	Create a file in the current directory.	From /home, creates /home/dockerfile.	Linux filenames are case-sensitive.
10	cat dockerfile	Display file contents.	Shows saved Dockerfile instructions.	Does not edit the file.
11	vi dockerfile	Open the Vi text editor.	Press i to insert; Esc then :wq and Enter to save/quit.	Use Esc before :wq.
12	exit	Exit current shell or interactive session.	May return to host shell or previous user.	Does not automatically remove a container.

3. Downloading and managing images

No.	Command	Theory	Example / expected result	Exam tip
13	docker pull nginx	Download Nginx image from a registry, usually Docker Hub.	Image layers download; image becomes local.	pull = download an image.
14	docker pull httpd	Download Apache HTTP Server image.	Image becomes available to create a container.	Nginx and Apache are web servers.
15	docker images	List local images.	Shows repository, tag, image ID and size.	Shows images, not running containers.

16	docker image ls	Alternative command to list local images.	Similar output to docker images.	Both are valid.
17	docker rmi nginx	Remove local nginx image reference if permitted.	May fail if a container depends on it.	Remove dependent containers first if appropriate.
18	docker images	Recheck local images after removal.	Verify whether nginx tag remains.	Check the command result.
19	docker image rm nginx	Alternative image-removal syntax.	Removes image reference when permitted.	rmi is short form of image rm.

4. Creating, viewing and accessing containers

No.	Command	Theory	Example / expected result	Exam tip
20	docker run -dt --name akash1 nginx	Create and start a named Nginx container in the background with a pseudo-terminal allocated.	docker ps should show akash1 if its main process stays running.	run can create and start.
21	docker run -dt --name akash2 httpd	Create and start an Apache HTTP Server container.	Check using docker ps.	--name sets a readable name.
22	docker ps	List running containers.	Shows ID, image, status, ports and name.	Stopped containers are hidden.
23	docker ps -a	List all containers, including stopped ones.	Shows running and exited containers.	-a = all.
24	docker ps -q	Show only IDs of running containers.	Useful in scripts.	-q = quiet.
25	docker stop akash1	Gracefully ask the main process to stop.	Container stops but remains available.	Stop is not remove.
26	docker start akash1	Start an existing stopped container.	Check with docker ps.	Does not create a new container.
27	docker restart akash1	Stop and start an existing	Useful after some	Writable

		container.	configuration changes.	layer remains.
28	docker rm akash1	Remove a stopped container.	Verify with docker ps -a.	Stop it first if running.
29	docker rm -f akash1	Force-remove a container, stopping it if needed.	Container is removed.	Can interrupt a running application.
30	docker exec -it akash1 bash	Start an interactive Bash shell in an already-running container.	Run pwd or ls inside it.	exec runs a process in an existing container.
31	exit	Leave the interactive shell.	Return to host shell.	Usually does not stop container.

5. Copying files between host and container

No.	Command	Theory	Example / expected result	Exam tip
32	touch /home/akash.txt	Create an empty file on the Ubuntu host.	File exists at /home/akash.txt.	This is a host file.
33	docker cp /home/akash.txt akash1:/home/	Copy host file into a container.	File appears at /home/akash.txt inside container.	Use container name or ID.
34	docker exec -it akash1 bash	Open shell to verify the copy.	Run ls /home inside container.	Container should be running.
35	docker cp akash1:/home/akash.txt /home/akash-copy.txt	Copy file from container to host.	Host receives /home/akash-copy.txt.	Source path must exist.

6. Port mapping

No.	Command	Theory	Example / expected result	Exam tip
36	docker run -d --name web1 -p 8080:80 nginx	Publish container port 80 on host port 8080.	Open http://localhost:8080 on the Docker host.	Syntax: -p HOST_PORT:CONTAINER_PORT.
37	docker run -dt -p 13:80 --name akash3 nginx	Map host port 13 to container port 80.	http://localhost:13 if allowed and reachable.	Port 13 is unusual; use 8080 for practice.
38	docker ps	Show running containers and published ports.	May show 0.0.0.0:8080->80/tcp.	Host 8080 forwards to container 80.
39	docker port web1	Display port mappings for a container.	May show 80/tcp -> 0.0.0.0:8080.	Useful for checking mappings.
40	docker logs web1	Display stdout/stderr logs from the main process.	Useful for startup troubleshooting.	Logs help diagnose failures.
41	docker inspect web1	Show detailed container configuration and state.	Includes network, mounts and port bindings.	Detailed JSON output.

7. Docker volumes — persistent storage

No.	Command	Theory	Example / expected result	Exam tip
42	docker volume create akashvolume	Create Docker-managed volume.	Docker returns the volume name.	Volume can exist without a container.
43	docker volume ls	List Docker volumes.	Shows akashvolume.	Lists volumes, not containers.
44	docker volume inspect akashvolume	Show volume metadata and mountpoint.	Useful for troubleshooting storage.	Docker Desktop paths may not be directly accessible.
45	docker run -dt --mount source=akashvolume,destination=/home --name akash1 nginx	Mount volume at /home in container.	Files created there are stored in volume.	source=volume; destination=container path.
46	docker exec -it akash1 bash	Open shell in running container.	Inspect or create files.	Container must be running.
47	cd /home	Move to mounted directory inside container.	/home is backed by the volume.	Path is inside container.
48	touch file1	Create an empty file in current directory.	Creates /home/file1 in volume.	Verify it with ls.
49	ls	List current directory contents.	Should show file1.	Verify before continuing.
50	docker stop akash1	Stop first container.	Container remains but is stopped.	Does not delete volume.
51	docker rm akash1	Remove first	Named volume	Key

		container.	remains unless separately deleted.	persistence lesson.
52	docker run -dt --mount source=akashvolume,destination=/mnt --name akash2 nginx	Mount same volume at /mnt in second container.	Previous file is at /mnt/file1.	One volume can be mounted at different paths.
53	docker exec -it akash2 bash	Open second container shell.	Inspect shared volume.	Container must be running.
54	cd /mnt	Move to mount destination in second container.	Access same volume as earlier.	Mount paths can differ.
55	ls	List volume contents.	Should show file1.	Proves data persisted.
56	docker volume rm akashvolume	Remove unused volume.	Deletes volume if permitted.	Can permanently delete stored data.

8. Docker networking — bridge and host

No.	Command	Theory	Example / expected result	Exam tip
57	docker network ls	List Docker networks.	Common defaults include bridge, host and none.	Lists networks.
58	docker network create --driver=bridge --subnet=192.168.0.0/24 akashbridge	Create custom bridge network with a subnet.	Creates akashbridge if subnet is available.	Subnet must not conflict with existing routes.
59	docker network inspect akashbridge	Show network configuration.	Displays subnet and connected containers.	Useful for troubleshooting.
60	docker run -dt --	Create container on	Container joins	--network

	network=akashbridge --name akash1 nginx	custom bridge network.	akashbridge.	selects network.
61	docker run -dt --network=akashbridge --name akash2 nginx	Create second container on same bridge.	Containers can normally resolve each other by name on a user-defined bridge.	Applications and ports must permit communication.
62	docker inspect akash1	Show container network settings.	Look under NetworkSettings for akashbridge.	Inspect includes IP and network details.
63	docker network connect akashbridge akash2	Connect existing container to network.	Container joins the network.	Both must exist.
64	docker network disconnect akashbridge akash2	Disconnect container from network.	Removes that network connection.	May remain connected to other networks.
65	docker run -dt --network=host --name akash3 nginx	Use host network namespace on supported Linux hosts.	No separate bridge IP is assigned.	Port publishing is ignored in host mode.

Bridge versus host network

Feature	Bridge network	Host network
Isolation	Separate container network	Shares host network namespace on

	interface/namespace.	supported Linux.
Container IP	Usually receives IP on bridge.	No separate bridge-network IP.
Communication	Same user-defined bridge containers can communicate where permitted.	Uses host network configuration.
Port mapping	-p 8080:80 can publish a container port.	-p is unnecessary and ignored in host mode.
Typical use	Web apps, APIs and databases in separate containers.	Applications needing direct host networking.

9. Dockerfile — build your own image

No.	Command / instruction	Theory	Example / expected result	Exam tip
66	cd /home	Move to directory where PDF creates Dockerfile.	File will be /home/dockerfile.	Use correct build directory.
67	touch dockerfile	Create empty file.	Creates file named dockerfile.	Conventional name is Dockerfile.
68	vi dockerfile	Edit file with Vi.	Type instructions and save.	Filename must match build command.
69	FROM ubuntu:latest	Select base image.	Starts with Ubuntu image.	Normally first instruction.
70	RUN apt update	Update package index during build.	Fetches available package information.	RUN executes at build time.
71	RUN apt upgrade -y	Upgrade installed packages non-interactively.	Packages may upgrade during build.	-y answers yes.
72	RUN apt install apache2 -y	Install Apache into image.	Image contains Apache files/config.	Does not ensure Apache starts at runtime.
73	RUN mkdir /dockerfile	Create directory in image filesystem.	/dockerfile exists in image.	mkdir = make

				directory.
74	RUN touch /dockerfile/akash.txt	Create empty file during build.	File exists in resulting image.	Not automatically a host file.
75	EXPOSE 77	Document intended container port.	Image metadata records port 77.	Does not configure or publish port.
76	:wq	Save and exit Vi.	Dockerfile is saved.	Press Esc first.
77	cat dockerfile	Display instructions.	Check saved content.	Check spelling.
78	docker build -t akashdockeraws/akashdocker1008:ubuntuimage .	Build image and tag it.	Creates named image if build succeeds.	Dot is current build context.
79	docker images	List local images.	Find repository and tag.	Confirm build succeeded.
80	docker image inspect akashdockeraws/akashdocker1008:ubuntuimage	Display image metadata.	Shows layers, configuration and exposed-port metadata.	Image inspect differs from container inspect.

10. Docker Hub — login, push and pull

No.	Command	Theory	Example / expected result	Exam tip
81	docker login	Authenticate with registry, normally Docker Hub.	Complete supported authentication flow.	Never share password or access token.
82	docker build -t akashdockeraws/akashdocker1008:ubuntuimage .	Build/tag image for intended repository.	Local image receives repository and tag.	Correct naming matters for push.
83	docker push	Upload tagged image	Layers upload if	Ensure

	akashdockeraws/akashdocker1008:ubuntuimage	to Docker Hub.	authentication and permissions are correct.	repository/ account permissions.
84	docker pull akashdockeraws/akashdocker1008:ubuntuimage	Download published image.	Another machine can pull it.	Pull = download; push = upload.
85	docker run -dt --name akash123 akashdockeraws/akashdocker1008:ubuntuimage	Create/start container from image reference.	Works if image exists and startup process keeps running.	Original Dockerfile may exit because Apache is not started.
86	docker ps	Check running containers.	Look for akash123.	If missing, use docker ps -a.
87	docker exec -it akash123 bash	Open shell in running container.	Inspect files and software.	Fails if container exited.
88	ls	List current directory.	Shows entries in current directory.	Use pwd to confirm location.
89	cd /dockerfile	Enter directory created during build.	Inspect /dockerfile/akash.txt.	Leading / means absolute path.
90	ls	List directory contents.	Should show akash.txt.	Verifies build step.
91	apache2 -v	Display Apache version.	Shows version if installed.	Checks installation, not runtime status.

Security tip: do not type secrets into Dockerfiles or bake passwords/access tokens into images. Use secure authentication methods.

11. Complete sequence in order :

Ste	Task	Command	What to verify

p			
1	Check Docker	docker --version	CLI is installed.
2	Download image	docker pull nginx	Image is downloaded.
3	List images	docker images	nginx appears locally.
4	Create container	docker run -d --name akash1 nginx	Container starts in background.
5	Check running containers	docker ps	akash1 appears if running.
6	Check all containers	docker ps -a	Running and stopped containers appear.
7	Open container shell	docker exec -it akash1 bash	Enter container.
8	Inspect files	ls /home	List files inside container.
9	Leave shell	exit	Return to host.
10	Publish web port	docker run -d --name web1 -p 8080:80 nginx	Host port 8080 published.
11	Test website	Open http://localhost:8080	Nginx welcome page should appear.
12	Create volume	docker volume create akashvolume	Persistent volume created.
13	List volumes	docker volume ls	Shows akashvolume.
14	Create network	docker network create --driver bridge akashbridge	Custom bridge created.
15	Inspect network	docker network inspect akashbridge	Shows network configuration.
16	Build image	docker build -t myapache:v1 .	Build from Dockerfile in current directory.
17	Log in	docker login	Authenticate with Docker Hub.
18	Push image	docker push YOUR_USERNAME/myapache:v1	Upload correctly tagged image you own.
19	Stop container	docker stop akash1	Container stops.
20	Remove container	docker rm akash1	Stopped container is removed.
21	Remove image	docker rmi nginx	Removes image reference if no longer in use.

12. Think Questions

Question	Solution
What is Docker?	A platform for building, distributing and running applications in containers.
What is a Docker image?	A template used to create containers.
What is a Docker container?	An isolated runtime instance of an image.
Difference between docker pull and docker push?	pull downloads an image; push uploads an image.
Difference between docker run and docker start?	run creates and starts a container; start starts an existing stopped container.
Difference between docker ps and docker ps -a?	ps lists running containers; ps -a lists all containers.
Why use docker exec -it?	To interactively execute a command inside a running container.
Why use volumes?	To keep data separate from a container's writable layer so it can survive container replacement.
What is port mapping?	Forwarding traffic from a host port to a container port.
What is a bridge network?	A network that allows connected containers to communicate.
What is a Dockerfile?	A text file with instructions for building a Docker image.
Purpose of FROM?	Specifies the base image.
Purpose of RUN?	Executes a command during image building.
Purpose of EXPOSE?	Documents an intended container port; it does not publish that port.
Purpose of CMD?	Specifies the default command to run when a container starts.
Purpose of docker inspect?	Displays detailed configuration and metadata.
Does docker rm delete a named volume automatically?	Normally no; named volumes have a separate lifecycle.
What does docker login do?	Authenticates with a Docker registry.

13. Final Table

If asked to…	Remember
Download an image	docker pull

Upload an image	docker push
Create and start a container	docker run
Start an existing container	docker start
Stop a container	docker stop
Remove a container	docker rm
Remove an image	docker rmi
List images	docker images
List running containers	docker ps
List all containers	docker ps -a
Enter a container	docker exec -it
Copy files	docker cp
Create persistent storage	docker volume create
List volumes	docker volume ls
Create a network	docker network create
Inspect network	docker network inspect
Build an image	docker build
Authenticate with Docker Hub	docker login
Show container details	docker inspect
Publish host port	-p HOST_PORT:CONTAINER_PORT

Questions and Answers :

- Part 1: Basic understanding of Docker

No.	Question	Easy Answer
1	What is Docker?	Docker is a tool that helps us run applications in separate, isolated environments called containers.
2	Why do we use Docker?	We use Docker to run applications without needing to install all their software directly on our computer.
3	What is a Docker image?	An image is a template used to create a container.
4	What is a Docker container?	A container is an environment created from an image where an application can run.
5	What is the difference between an image and a container?	An image is like a recipe; a container is like the dish prepared using that recipe.

6	What is Docker Hub?	Docker Hub is an online place where we can store, download, and share Docker images.
7	What is Docker Engine?	Docker Engine is the software that manages Docker containers, images, networks, and volumes.
8	What is a Dockerfile?	A Dockerfile is a text file containing instructions for creating a Docker image.
9	What is Nginx?	Nginx is software commonly used to serve website and handle web traffic.
10	What is Apache HTTP Server?	Apache is software that can serve websites to users through a browser.
11	What is a repository in Docker Hub?	A repository is a place where related versions of an image are stored.
12	What is an image tag?	A tag is a label used to identify a particular version or variant of an image.

Part 2: Basic Linux questions

No.	Question	Easy Answer
13	What is Linux?	Linux is an operating system used on servers, computers, and cloud machines.
14	What is Ubuntu?	Ubuntu is a popular operating system based on Linux.
15	What is a terminal?	A terminal is a place where we type commands to control a computer.
16	What is a shell?	A shell reads our commands and asks the operating system to perform them.
17	What is the root user?	Root is the administrator account with extensive permissions.
18	What does sudo su - do?	It switches to the root user and loads the root login environment.
19	What does sudo su do?	It switches to the root user without the same full login-environment setup as sudo su -.
20	What does whoami do?	It tells us which user we are currently using.
21	What does pwd do?	It tells us the directory we are currently working in.
22	What does cd do?	It changes our current directory.
23	What does cd /home do?	It moves us to the /home directory.
24	What does cd / do?	It moves us to the top-level directory of the Linux filesystem.
25	What does ls do?	It shows the files and folders in a directory.
26	What does touch akash.txt do?	It creates an empty file called akash.txt if the file does not exist.
27	What does cat dockerfile do?	It displays the contents of the file named dockerfile.
28	What does vi dockerfile do?	It opens the file in the Vi text editor so we can edit it.
29	What does :wq mean in Vi?	It saves the changes and quits the editor.
30	What does exit do?	It exits the current shell or interactive session.

Part 3: Docker image questions		
No.	Question	• Easy Answer
31	What does docker pull nginx do?	It downloads the Nginx image to our computer.
32	What does docker pull httpd do?	It downloads the Apache HTTP Server image.
33	What does docker images do?	It displays the Docker images available on our computer.
34	What does docker image ls do?	It lists the images stored locally, just like docker images.
35	What does docker rmi nginx do?	It removes the local Nginx image reference if Docker allows it.
36	Can we remove an image that a container is using?	Docker may refuse to remove the image while a container depends on it.
37	What is the difference between docker pull and docker rmi?	pull downloads an image; rmi removes a local image reference.
38	How do we check whether an image was downloaded?	Run docker images and look for the image name.

Part 4: Docker container questions		
No.	Question	Easy Answer
39	What does docker run do?	It creates a new container from an image and starts it.
40	What does docker run -dt -name akash1 nginx do?	It creates and starts a container named akash1 using the Nginx image in the background.
41	What does -d mean?	It runs the container in the background.
42	What does -t mean?	It allocates a pseudo-terminal for the container process.
43	What does --name akash1 mean?	It gives the container the name akash1.
44	What does docker run -dt --name akash2 httpd do?	It creates and starts a container named akash2 using the Apache HTTP Server image.
45	What does docker ps do?	It shows containers that are currently running.
46	What does docker ps -a do?	It shows all containers, including stopped containers.
47	What does docker ps -q do?	It displays only the IDs of running containers.
48	What does docker stop akash1 do?	It stops the running container named akash1.
49	What does docker start akash1 do?	It starts an existing stopped container named akash1.
50	What does docker restart akash1 do?	It stops and starts the existing container again.
51	What does docker rm akash1 do?	It removes the stopped container named akash1.
52	What does docker rm -f akash1 do?	It force-removes the container, stopping it if necessary.
53	What does docker exec -it	It opens an interactive Bash shell inside the

	akash1 bash do?	running container.
54	What does docker inspect akash1 do?	It displays detailed information about the container.
55	What is the difference between docker stop and docker rm?	Stop stops a container; rm removes the container.
56	What is the difference between docker run and docker start?	run creates and starts a new container; start starts an existing stopped container.
57	What happens when we type exit inside a container shell?	We leave the interactive shell and return to the previous shell. The container usually continues running.
58	What should we do if a container does not appear in docker ps?	Run docker ps -a to check whether it has stopped or exited.

Part 5: Copying files between Linux and Docker		
No.	Question	Easy Answer
59	What does docker cp do?	It copies files or directories between the host computer and a container.
60	What is the host machine?	It is the computer or server on which Docker is running.
61	What does touch /home/akash.txt do?	It creates an empty file at /home/akash.txt on the host machine.
62	How do we copy a host file into a container?	Use docker cp /home/akash.txt akash1:/home/.
63	What does akash1:/home/ mean?	It means the /home/ directory inside the container named akash1.
64	How do we copy a file from a container to the host?	Use docker cp akash1:/home/akash.txt /home/akash-copy.txt.
65	Why do we use docker cp?	It helps us transfer files without needing to build a new image.

Part 6: Docker port mapping		
No.	Question	Easy Answer
66	What is a port?	A port is a numbered communication endpoint used by applications to send and receive network traffic.
67	What is port mapping in Docker?	It connects a port on the host machine to a port inside a container.
68	What does -p 8080:80 mean?	Host port 8080 forwards traffic to container port 80.
69	What does docker run -d --name web1 -p 8080:80 nginx do?	It starts Nginx and publishes its container port 80 on host port 8080.
70	How can we open the Nginx website on the same computer?	Open http://localhost:8080 in a browser if the container and port mapping are working.
71	What does docker run -dt -p 13:80 --name akash3 nginx do?	It maps host port 13 to port 80 inside the Nginx container.
72	What does docker port web1 do?	It displays the port mappings for the container named web1.
73	What does docker logs web1 do?	It displays logs produced by the container's main process.
74	Does EXPOSE 80 publish a port automatically?	No. EXPOSE documents an intended container port. Use -p to publish a port.
75	Why might a website fail to open?	The container may be stopped, the application may not be running, the port may be wrong, or a firewall may block access.

Part 7: Docker volume questions		
No.	Question	Easy Answer
76	What is a Docker volume?	A volume is storage managed by Docker that can keep data even after a container is removed.
77	Why do we use volumes?	To keep important data separate from the container so it can survive container replacement.
78	What does docker volume create akashvolume do?	It creates a volume named akashvolume.
79	What does docker volume ls do?	It lists the Docker volumes.
80	What does docker volume inspect akashvolume do?	It displays detailed information about the volume.
81	What does --mount source=akashvolume,destination=/home mean?	It connects the volume akashvolume to the /home directory inside the container.
82	What does touch file1 do inside the mounted directory?	It creates an empty file named file1 in the volume-backed directory.
83	What happens to a named volume when we remove its container?	Normally, the volume remains available.
84	Can two containers use the same volume?	Yes. They can access the same stored data when the volume is mounted in both containers.
85	What does destination=/mnt mean?	The volume is mounted at the /mnt directory inside the container.
86	If one container saves file1 in a volume, can	Yes, if the second container

	another container see it?	mounts the same volume and looks in the correct directory.
87	What does docker volume rm akashvolume do?	It removes the volume if it is not in use. Its stored data may be permanently lost.
88	What is the difference between a container and a volume?	A container runs an application; a volume stores data.
89	How can we prove that a volume preserves data?	Create a file in one container, remove that container, mount the same volume in another container, and check whether the file still exists.

Part 8: Docker networking questions		
No.	Question	Easy Answer
90	What is Docker networking?	It allows containers to communicate with other containers and, when configured, external systems.
91	What does docker network ls do?	It lists the Docker networks.
92	What does docker network create --driver=bridge --subnet=192.168.0.0/24 akashbridge do?	It creates a bridge network named akashbridge with the specified subnet, if the subnet is available.
93	What is a bridge network?	It is a Docker network that connects containers through a virtual network.
94	What does docker network inspect akashbridge do?	It displays the configuration and connected containers of that network.

95	What does --network=akashbridge mean?	It connects a container to the network named akashbridge.
96	Can two containers on the same user-defined bridge network communicate?	Yes, normally they can communicate if their applications, ports, and network rules allow it.
97	What does docker network connect akashbridge akash2 do?	It connects the existing container akash2 to the network akashbridge.
98	What does docker network disconnect akashbridge akash2 do?	It disconnects akash2 from akashbridge.
99	What is host networking?	It allows a container to share the host's network namespace on supported Linux systems.
100	What does --network=host do?	It runs the container using host networking instead of a separate bridge network.

Part 9: Dockerfile questions		
No.	Question	Easy Answer
104	What is a Dockerfile?	It is a text file that tells Docker how to build an image.
105	Why do we use a Dockerfile?	To build images automatically using written instructions.
106	What does FROM ubuntu:latest mean?	It selects the Ubuntu image as the starting point for building our image.
107	What does RUN do?	It executes a command during the image build process.

108	What does RUN apt update do?	It updates Ubuntu's package information so available software can be found.
109	What does RUN apt upgrade -y do?	It upgrades installed packages and automatically answers yes to confirmation prompts.
110	What does RUN apt install apache2 -y do?	It installs Apache HTTP Server in the image.
111	What does RUN mkdir /dockerfile do?	It creates a directory named /dockerfile inside the image.
112	What does RUN touch /dockerfile/akash.txt do?	It creates an empty file named akash.txt inside that directory.
113	What does EXPOSE 77 do?	It documents that the image is intended to use container port 77. It does not make Apache listen on that port.
114	What does CMD do?	It specifies the default command that runs when a container starts.
115	What is the difference between RUN and CMD?	RUN executes during image building; CMD specifies the default command for container startup.
116	What does docker build do?	It builds a Docker image using a Dockerfile and build context.
117	What does -t mean in docker build -t myapache:v1 .?	It assigns a name and tag to the image.
118	What does the dot (.) at the end of docker build mean?	It specifies the current directory as the build context.

119	What does docker build -t akashdockeraws/akashdocker1008:ubuntuimage . do?	It builds an image and tags it with the repository name akashdockeraws/akashdocker1008 and tag ubuntuimage.
120	Why might a container stop immediately after starting?	Its main process may finish or fail. For example, installing Apache alone does not guarantee that Apache starts.
121	How can we check the Apache version?	Run apache2 -v inside an environment where Apache is installed.
	Does apache2 -v prove that Apache is running?	No. It displays version information, not the service's running status.

Part 10: Docker Hub questions		
No.	Question	Easy answer
123	What does docker login do?	It authenticates us with Docker Hub or another supported Docker registry.
124	Why do we log in to Docker Hub?	To authenticate before performing operations that require account access, such as pushing an image.
125	What does docker push do?	It uploads a tagged image to a registry.
126	What does docker pull do?	It downloads an image from a registry.

127	What does docker push akashdockeraws/akashdocker1008:ubuntuimage do?	It attempts to upload that tagged image to the specified Docker Hub repository.
128	What must be correct before pushing an image?	The image tag, registry authentication, repository permissions, and repository name must be correct.
129	What does akashdockeraws/akashdocker1008:ubuntuimage mean?	akashdockeraws is the account or namespace, akashdocker1008 is the repository, and ubuntuimage is the tag.
130	What is the difference between docker build and docker push?	build creates an image locally; push uploads a tagged image to a registry.
131	How can another computer use our Docker Hub image?	It can pull the image and create a container from it, provided it has access to the image.
	Why should we not store passwords in a Dockerfile?	They could become exposed through the image or its build history. Use secure authentication methods instead.